

HANG MAN

A Report On Mini Project

B.sc.

in

Department of Computer Science

by

Sreehari MA (CCAXSCS026)

Madhav M (CCAXSCS047)

Athul PS (CCAXSCS036)

Alfina M (CCAXSCS018)

Under the supervision of

Ms. BLESSY GEORGE M



to

**DEPARTMENT OF COMPUTER SCIENCE
CHRIST COLLEGE(AUTONOMOUS), IRINJALAKUDA
IRINJALAKUDA-680125, INDIA**

August 2024

Abstract

A hangman is a traditional word-guessing game that is HTML, CSS, and JavaScript implemented. Players who play this version try to reveal the hidden word by picking one letter at a time. The tension builds up as you get wrong and wrong with each incorrect guess drawn on the gallows. In this game, players can click on letters to make guesses; it has an interactive interface as well as visual progress of the hangman's drawing. This online variant of Hangman provides users with a straightforward design and easy-to-understand rules for all people irrespective of their age group.

Contents

1	Introduction	4
1	Background	4
2	Objectives	4
3	Scope	5
2	Literature Review	7
1	Related Work	7
1.1	Basic Hangman Game	7
1.2	Hangman Game with a Dictionary API	8
2	Technologies Used	8
3	System Design	10
1	Overview	10
1.1	HTML (Structure):	10
1.2	CSS (Styling):	11
1.3	JavaScript (Functionality):	11
1.4	Uses:	12
2	Architecture	12

2.1	Frontend Architecture:	12
2.2	System Components:	15
3	Module Descriptions	17
3.1	HTML (index. html)	17
3.2	CSS (styles. css)	18
3.3	JavaScript (script. js)	18
4	User Interface Design	19
4	Implementation	22
1	Development Tools	22
2	Coding Standards	22
3	Code Implementation	23
4	Challenges Faced	23
5	Testing	24
1	Testing Strategies	24
2	Test Cases	24
3	Test Results	25
6	Conclusion and Future Work	26
1	Conclusion	26
2	Future Enhancements	26
7	References	27
A	Appendices	29
1	Appendix A: Source Code	29

1.1	HTML Code:	29
1.2	CSS Code:	30
1.3	JAVASCRIPT Code:	35
2	Appendix B: User Manual	43
2.1	Starting:	43
2.2	Game Progress:	43
2.3	Understanding Feedback:	43

Chapter 1

Introduction

1 Background

HTML, CSS, and JavaScript are used to develop a browser-based version of the classic hangman word game. HTML is employed in building the game interface while CSS gives it a visual appeal and JavaScript takes care of the game logic. The players try to guess words by suggesting letters so that each incorrect answer brings them closer to “hanging” a stick figure. This often involves some basic way of inputting letters, live updates on right or wrong answers and showing a picture which depicts a hanged man as he makes mistakes along the way. It means that one can play an interactive responsive game through their browser thanks to these technologies working together.

2 Objectives

These are the aims for creating Hangman game in HTML, CSS and JavaScript:

- Make a basic browser based Hangman game which is interactive.
- Develop the structure of the game using HTML to display the word, provide buttons for alphabets and also include a picture of hanged man.
- Ensure that CSS styles are used to make the game look nice and friendly.
- Use Javascript to create logic for word selection, user input, progress of the game as well as win/lose conditions
- Make it compatible with different devices and screen sizes by ensuring responsive design.
- It should show when you get something right or wrong on the display and how many attempts are left.

3 Scope

A Hangman game developed in HTML, CSS, and JavaScript is where a dynamic web-based application will be developed in which a player tries to guess a word by guessing letters. The HTML part defines the structure of the game, which holds elements such as word display, guessed letters, and hangman figure. These elements are then styled using CSS to produce an enriched layout that self-adjusts according to the size of different screens. JavaScript provides game logic for selecting a random word, updating the display based on correct letters that were guessed, tracking wrong guesses, and win/loss conditions. It also offers a few user-interaction features with

respect to input fields or buttons to provide the guesses, and feedback mechanisms telling the player how well they do and what attempts have been used.

Chapter 2

Literature Review

1 Related Work

1.1 Basic Hangman Game

1.Description: This will be the easiest implementation of a Hangman game, done with basic HTML, CSS, and JavaScript. It's a letter-guessing game to complete the word, where each incorrect guess results in the drawing of part of the hangman.

2.Features:

- A list of basic words within JavaScript.
- A hangman figure that is drawn based on incorrect guesses.
- The design is responsive for various screen sizes.

3.Learning Points:

- How to update the word and hangman figure by manipulating the DOM?
- Event Listeners to capture user input.

1.2 Hangman Game with a Dictionary API

1.Description: A more challenging Hangman game that draws its words from an external dictionary API.

2.Features:

- The words are chosen by fetching random words through the integrated dictionary API.
- After the game is over, display the meaning of the word. Option for difficulty levels.

3.Learning Points:

- Working with APIs; handling asynchronous requests; error handling for API calls.
- It utilizes JavaScript promises and Fetch API.

2 Technologies Used

- Structure: Determines the layout of each kind of game and specifies such components as buttons and text zones on the Web site's page.

- Game Elements: Responsible for generating the interface components such as the box/sLOT for the word to be guessed the hangman figure drawing and the buttons for the user's choices.
- CSS (Cascading Style Sheets): Styling: This is the aspect of the game that deals with how it looks, starting from the screen layout, the color used, the font used and the general feel of the game.
- Responsive Design: Makes sure that the game nicely fits the screen of any device and does not look like it was stretched or shrunk.
- Animations: Used for styling of such items as the hangman drawing or the win/lose

Chapter 3

System Design

1 Overview

1.1 HTML (Structure):

- Game Layout: Creates the layout of the game that will contain the word to guess, letters and attempts left.
- Input Fields: Presents text-input fields through which the users can input their guesses.
- Buttons: Consists of links that you can press to participate in the game, for example, to submit a guess or to start a new round of the game.
- Game Status: Shows the current count of attempts and also the current word displayed on the screen for the user to guess.

1.2 CSS (Styling):

- Visual Design: Decorates the game elements, to include font, color and lay out to suit the desires of the user.
- Game Board: Adjusts the visuals of the game board which involves the word, letters that the player guessed, and the number of attempts remaining in the game.
- Interactive Elements: Makes buttons and input fields more distinguishable and visually friendly to the users.

1.3 JavaScript (Functionality):

- Game Logic: Contains all the real-game-specific core logic such as picking a random word, checking the guesses, and updating the game's state.
- Event Handling: Facilitates the user, interacting with the system like a button click or input of a guess.
- State Management: It keeps and modifies the game status: the word to guess, the correct and incorrect attempts and the number of attempts left.
- User Feedback: Communicates with the user, e.g., enable the user to see which letters have been guessed; present game over messages; update the display according to the user's actions.

1.4 Uses:

- Educational Tool: Allows users to comprehend fundamental concepts of programming like entering/reading user input, translating game logic, and updating the graphical UI display.
- Skill Practice: Facilitates the application of the html, css, and javascript all in one project thus enabling the developers to train.
- Entertainment: Contributes to active and cooperative user engagement as a result of coming up with a game that is entertaining to play and at the same time solving a puzzle.
- Prototype: Can be used as a prototype to introduce more complicated game or application.

2 Architecture

2.1 Frontend Architecture:

HTML Structure:

- Container: Primary container for all other game related object or component.
- Options Container: Shows the names of the categories to be selected to search for words in this category.
- Letter Container: The buttons that demonstrate the letters in alphabet for guessing the letters.

- User Input Section: Displays the solution to the level, where the actual word is covered with underscores and letters that the user has guessed right.
- Canvas: Location of the word which is used to draw the hangman figure when the correct guesses are wrong.
- New Game Popup: Display results of the game and enables one to play a new game.

CSS Styling:

- General Styles: Identifies the grid and flow for placing objects and content, typeface, and fundamental look of the object.
- Button Styles: Permits to control the look and the functions of the option and letter buttons.
- Letter Container Styles: Responsible for letter's buttons, and their appearance.
- Canvas Styles: Appends border and style to the canvas element.
- Popup Styles: Arranges for styling of the result popup, for example, by deciding on the position of the banner in relation to the functional design of the application.

JavaScript Functionality:

1.Initialization (initializer Function):

- Creates the whole atmosphere of the game.
- Creates and shows the option and letter buttons.
- Starts the game by creating variables that are later used for the game and minimizes things that are not needed in the game.

2.Option Selection (displayOptions and generateWord Functions):

- displayOptions: Gives contextual buttons for different word categories and produces them.
- generateWord: Picks a random word from the selected category, and, consequently, changes the appearance of the user interface.

3.Letter Guessing:

- Handles letter button clicks.
- Collects correct guesses for updating the display and counts invalid guesses made.
- Handles the win/lose state and influences the game at the current state.

4.Drawing on Canvas (canvasCreator Object and drawMan Function):

- canvasCreator: Offers ways to write functions that will set the various parts of the hangman figure such that they include the head, body, arms, and legs.
- drawMan: Hangs the figure on the canvas in relation to the wrong number of guesses served as the input.

5.Game End (blocker Function):

- Closes the board and disables all the buttons (note: option buttons and letters) at the end of the gam
- Shows the result in the new game popup screen.

6.New Game (newGameButton Event Listener):

- It reloads the game and creates a new game if clicked.

2.2 System Components:

1.HTML Elements: Areas such as the design of the game interfaces and their construction.

2.CSS: Supplies them for appearance and design to make the interface friendly to the users.

3.Canvas: DRAW used to create the hangman figure in dynamic drawing.

4.Game Logic:

- Word Selection: Picks an image of a randomly selected word from the category that has been chosen.
- Guess Management: Responsible for letter guesses, display update and counters wrong guesses.
- Win/Loss Conditions: Establishes whether it is the victory or defeat for player depending on the guesses.

5.State Management:

- Game State Variables: Stores variables for the current word in the game, total incorrect guesses, and the state of winning/losing.
- UI Updates: Draws new frames of the hanged man, displays letters that are in the word, etc.

6.Event Handling:

- Button Clicks: Oversees the responses of the user to the option and letter buttons.
- New Game Button: Expires when clicked and begins a new round of the game.

7.Rendering:

- Dynamic Content Update: Modifies the GUI as a consequence of the user interactions such as; writing letters, drawing the hangman figure.
- Canvas Rendering: Paints the figures of a man hanged on the canvas depending on the number of times the root has been guessed incorrectly.

8.Summary:

- Frontend Architecture: Structural: HTML Structural: CSS, and for game logic and interactions: JavaScript.
- System Components: Comprises the view, the model, controller, the action and logic, responding system and graphical output.

- Integration: JavaScript performs the evident design of the game in terms of the User Interface and interactions with the layout updating the game as soon as the user interacts with it.

3 Module Descriptions

3.1 HTML (index. html)

1.Structure: Prints out the framework of the game, where components for the word to be guessed, the letters that have been guessed, and the man hanged are situated.

2.Elements:

- div(word): Shows the current state of the word that is being guessed with underscores for the blank spaces.
- div(guessed-letters): Displays letters that have been previously guessed.
- div(hangman-image): Shows the current status of the hangman being drawn on the board.
- input(guess): Category entered by players for them to input/enter their guess.
- button(submit): A button by which a user inputs the guessed letter of a word.
- div(message): Such messages as win, lose, or guess is invalid should be displayed.

3.2 CSS (styles. css)

1.Styling: Responsible for the artistic creation of the game such that it presents layout, fonts, color scheme, and positioning of objects.

2.Elements:

- Perhaps the layouts for word, guessed-letters, and hangman-image to make them stand out and look proper.
- Adaptive design in a sense that the game that is created should look good when viewed on any device.

3.3 JavaScript (script. js)

1.Logic:

- Initialization: Delivers the list of words, chooses a word randomly, and defines the game's parameters.
- Game Mechanics: Cleans the user's input, redraws the screen with the letters of the typed word and hangman figure, and checks for a win/loss condition.
- Event Listeners: Aware of the user's move like the submission of a guess or the initiation of a new round.
- Hangman Image Update: Screenshots the hangman logo according to the number of wrongly guessed letters.

2.Features:

- Word Selection: Uses a random value with its arguments from a fixed list of words.
- Guess Tracking: Responsible for tabulating the correct and the wrong answers.
- Hangman Visualization: Updates the hangman image to the one depending on the number of the wrong guesses.
- Win/Loss Messages: Shows messages that are called on victory or defeat of the game.
- Input Validation: Validates the current input for if it is a single letter, and if that letter has already been guessed.

3. Dependencies:

- None: The game is coded using regular HTML and JavaScript; I did not use any HTML5 features or use of the jQuery library.

4 User Interface Design



Figure 3.1: Main Menu

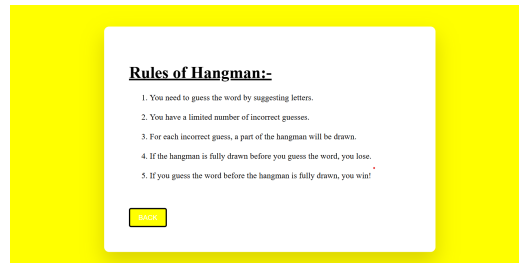


Figure 3.2: Rules Page

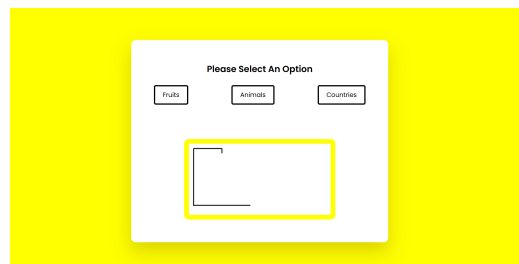


Figure 3.3: Starting Section

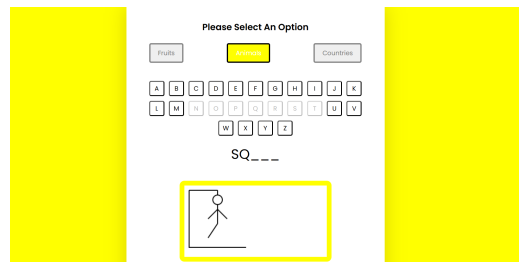


Figure 3.4: Game Section

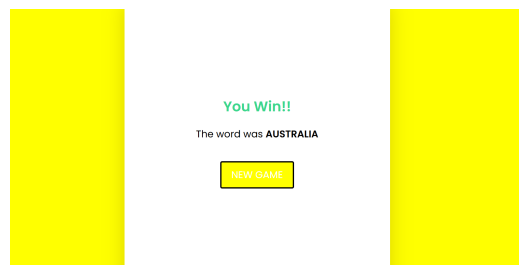


Figure 3.5: Game Won

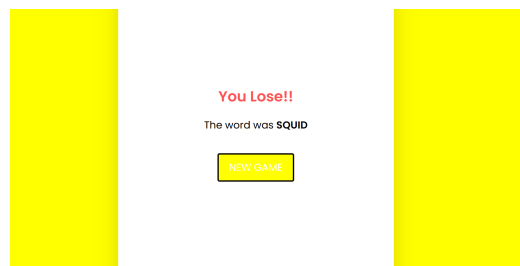


Figure 3.6: Game Lost

Chapter 4

Implementation

1 Development Tools

The tools that are directly used during the development process are a text editor, for example, VS Code, a browser for testing purposes, version control by means of Git, etc.

2 Coding Standards

- Proper use of names for the variables and functions in order to avoid confusion.
- Client-server code design to separate the business logic layer from the application's viewing layer.
- Adding comment and document for clarity and easier to maintain.

3 Code Implementation

JavaScript is used for developing the game logic, HTML and CSS which are used for providing structure and aesthetic look. These include the selection of words to be inputted, validation of the inputs as well as the updating of the state of the game.

4 Challenges Faced

- Adapting to the feature by using the concept of ‘Responsible Design’ so that the game is compatible with the various sizes of the screens.
- Dealing with special scenarios, including consecutive input and inputs that are not letters of the included characters.

Chapter 5

Testing

1 Testing Strategies

- Unit Testing: To confirm that the functions and specific modules of the designated individual are working smoothly, they are tested.
- Integration Testing: It describes some interactive method of testing between different modules in the software.
- User Testing: The how is: Responses gained from the users in order to enhance and bring better game results.

2 Test Cases

- Check if right work was produced, and shown.
- Input sanitization and game state changes.

- Review the activities' mobile-friendliness when testing the design on different devices.

3 Test Results

Thus, all test cases regarding the game are successfully fulfilled: the game works butter-smooth and provides correct responses to the user's guesses.

Chapter 6

Conclusion and Future Work

1 Conclusion

The Hangman game project captured shows the necessity of the plays use of HTML, CSS, and JavaScript in the development of a web application. Thus, the goal and objectives set in the project have been fulfilled to the extent that Project is functional and has an aesthetically good game.

2 Future Enhancements

The suggestion for improvement includes inclusion of levels, inclusion of scores, and incorporation of more words. It is also possible to think about the further evolution of the game towards an online, multiplayer one.

Chapter 7

References

1. MDN Web Docs - Hangman Game Example

- Link: [MDN Hangman Game Tutorial](#)
- Description: In this tutorial, Mozilla Developer Network walks through making a basic game of Hangman using HTML, CSS, and JavaScript. It's an excellent resource for beginners.

2. FreeCodeCamp - Build a Hangman Game

- Link: [FreeCodeCamp Hangman Game](#)
- Description: A step-by-step tutorial for making a Hangman game using JavaScript, detailing explanations and code snippets. This is appropriate for those who have some background in JavaScript.

3. CodePen — Hangman Game Demos

- Link: [CodePen Hangman Game](#)

- Description: CodePen has interactive demos of Hangman games, usually with source code that is accompanied. This will be an excellent place to get ideas of how others implemented the game and even experiment with the code.

4. YouTube - Hangman Game Tutorials

- Link: [YouTube Hangman Game](#)
- Description: There are so many YouTube tutorials going through the process of creating a Hangman game using HTML, CSS, and JavaScript. These tutorials may be anywhere from beginner to middle-of-the-road in their level.

5. How to Build Hangman with JavaScript — CSS-Tricks

- Link to the tutorial: [CSS-Tricks' Hangman Game Tutorial](#)
- Description: Through the article of CSS-Tricks, you will learn how to create a Hangman game focusing on JavaScript logic and CSS styles so that the game might look interactive and nice.

6. W3Schools - JavaScript Projects

- Link: [W3Schools JavaScript Hangman](#)
- Description: W3Schools provides a very basic implementation of Hangman. It is part of their series of JavaScript projects and thus is a simple implementation, good for getting started.

Appendix A

Appendices

1 Appendix A: Source Code

1.1 HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>Hangman</title>
<link href="https://fonts.googleapis.com/css2?
family=Poppins:wght@400;600&display=swap" rel="stylesheet"/>
<link rel="stylesheet" href="style.css" />
</head>
<body>
<div class="container">
```

```

<div id="options-container"></div>
<div id="letter-container" class="letter-container hide"></div>
<div id="user-input-section"></div>
<canvas id="canvas"></canvas>
<div id="new-game-container" class="new-game-popup hide">
<div id="result-text"></div>
<button id="new-game-button">NEW GAME</button>
</div>
</div>
<script src="script.js"></script>
</body>
</html>

```

1.2 CSS Code:

```

* {
  padding: 0;
  margin: 0;
  box-sizing: border-box;
  font-family: "Poppins", sans-serif;
}
body {
  background-color: yellow;
}
.container {

```



```

    font-size: 16px;
    background-color: #ffffff;
    width: 90vw;
    max-width: 34em;
    position: absolute;
    transform: translate(-50%, -50%);
    top: 50%;
    left: 50%;
    padding: 3em;
    border-radius: 0.6em;
    box-shadow: 0 1.2em 2.4em rgba(111, 85, 0, 0.25);
}
#options-container {
    text-align: center;
}
#options-container div {
    width: 100%;
    display: flex;
    justify-content: space-between;
    margin: 1.2em 0 2.4em 0;
}
#options-container button {
    padding: 0.6em 1.2em;
    border: 3px solid #000000;
    background-color: #ffffff;

```

```

    color: #000000;
    border-radius: 0.3em;
    text-transform: capitalize;
    cursor: pointer;
}
#options-container button:disabled {
    border: 3px solid #808080;
    color: #808080;
    background-color: #efefef;
}
#options-container button.active {
    background-color: yellow;
    border: 3px solid #000000;
    color: #fff;
}
.letter-container {
    width: 100%;
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: 0.6em;
}
#letter-container button {
    height: 2.4em;
    width: 2.4em;

```

```

    border-radius: 0.3em;
    background-color: #ffffff;
    cursor: pointer;
}
.new-game-popup {
    background-color: #ffffff;
    position: absolute;
    left: 0;
    top: 0;
    height: 100%;
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-direction: column;
    border-radius: 0.6em;
}
#user-input-section {
    display: flex;
    justify-content: center;
    font-size: 1.8em;
    margin: 0.6em 0 1.2em 0;
}
canvas {
    display: block;

```

```

    margin: auto;
    border: 10px solid yellow;
    border-radius: 10px;
}
.hide {
    display: none;
}
#result-text h2 {
    font-size: 1.8em;
    text-align: center;
}
#result-text p {
    font-size: 1.25em;
    margin: 1em 0 2em 0;
}
#result-text span {
    font-weight: 600;
}
#new-game-button {
    font-size: 1.25em;
    padding: 0.5em 1em;
    background-color: yellow;
    border: 3px solid #000000;
    color: #fff;
    border-radius: 0.2em;
}

```

```

        cursor: pointer;
    }
    .win-msg {
        color: #39d78d;
    }
    .lose-msg {
        color: #fe5152;
    }

```

1.3 JAVASCRIPT Code:

```

const letterContainer = document.getElementById("letter-container");
const optionsContainer = document.getElementById("options-container");
const userInputSection = document.getElementById("user-input-section");
const newGameContainer = document.getElementById("new-game-container");
const newGameButton = document.getElementById("new-game-button");
const canvas = document.getElementById("canvas");
const resultText = document.getElementById("result-text");
let options = {
    fruits: [
        "Apple",
        "Banana",
        "Mango",
        "Pineapple",
        "Grapes",

```

```

    "Watermelon",
  ],
  animals: [
    "Hedgehog",
    "Squid",
    "Squirrel",
    "Panther",
    "Walrus",
    "Zebra"
  ],
  countries: [
    "India",
    "Hungary",
    "Romania",
    "Switzerland",
    "West Indies",
    "Australia",
  ],
};
let winCount = 0;
let count = 0;
let chosenWord = "";
const displayOptions = () => {
  optionsContainer.innerHTML += <h3>Please Select An Option</h3>;
  let buttonCon = document.createElement("div");

```

```

for (let value in options) {
  buttonCon.innerHTML += <button class="options"
  onclick="generateWord( '${value}')">${value}</button>;
}
optionsContainer.appendChild(buttonCon);
};

const blocker = () => {
  let optionsButtons = document.querySelectorAll(".options");
  let letterButtons = document.querySelectorAll(".letters");
  optionsButtons.forEach((button) => {
    button.disabled = true;
  });
  letterButtons.forEach((button) => {
    button.disabled = true;
  });
  newGameContainer.classList.remove("hide");
};

const generateWord = (optionValue) => {
  let optionsButtons = document.querySelectorAll(".options");
  optionsButtons.forEach((button) => {
    if (button.innerText.toLowerCase() === optionValue) {
      button.classList.add("active");
    }
    button.disabled = true;
  });
};

```

```

letterContainer.classList.remove("hide");
userInputSection.innerText = "";
let optionArray = options[optionValue];
chosenWord = optionArray[Math.floor(Math.random()
* optionArray.length)];
chosenWord = chosenWord.toUpperCase();
let displayItem = chosenWord.replace(/./g,
'<span class="dashes">_</span>');
userInputSection.innerHTML = displayItem;
};
const initializer = () => {
winCount = 0;
count = 0;
userInputSection.innerHTML = "";
optionsContainer.innerHTML = "";
letterContainer.classList.add("hide");
newGameContainer.classList.add("hide");
letterContainer.innerHTML = "";
for (let i = 65; i < 91; i++) {
let button = document.createElement("button");
button.classList.add("letters");
button.innerText = String.fromCharCode(i);
button.addEventListener("click", () => {
let charArray = chosenWord.split("");
let dashes = document.getElementsByClassName("dashes");

```



```

    if (charArray.includes(button.innerText)) {
    charArray.forEach((char, index) => {
    if (char === button.innerText) {
    dashes[index].innerText = char;
    winCount += 1;
    if (winCount === charArray.length) {
    resultText.innerHTML = <h2 class='win-msg'>You Win!!</h2>
    <p>The word was <span>${chosenWord}</span></p>;
    blocker();
    }
    }
    });
    } else {
    count += 1;
    drawMan(count);
    if (count === 6) {
    resultText.innerHTML = <h2 class='lose-msg'>You Lose!!</h2>
    <p>The word was <span>${chosenWord}</span></p>;
    blocker();
    }
    }
    button.disabled = true;
    });
    letterContainer.append(button);
    }

```

```

displayOptions();
let { initialDrawing } = canvasCreator();
initialDrawing();
};
const canvasCreator = () => {
let context = canvas.getContext("2d");
context.beginPath();
context.strokeStyle = "#000";
context.lineWidth = 2;
const drawLine = (fromX, fromY, toX, toY) => {
context.moveTo(fromX, fromY);
context.lineTo(toX, toY);
context.stroke();
};
const head = () => {
context.beginPath();
context.arc(70, 30, 10, 0, Math.PI * 2, true);
context.stroke();
};
const body = () => {
drawLine(70, 40, 70, 80);
};
const leftArm = () => {
drawLine(70, 50, 50, 70);
};

```

```

const rightArm = () => {
drawLine(70, 50, 90, 70);
};
const leftLeg = () => {
drawLine(70, 80, 50, 110);
};
const rightLeg = () => {
drawLine(70, 80, 90, 110);
};
const initialDrawing = () => {
context.clearRect(0, 0, context.canvas.width, context.canvas.height);
drawLine(10, 130, 130, 130);
drawLine(10, 10, 10, 131);
drawLine(10, 10, 70, 10);
drawLine(70, 10, 70, 20);
};
return { initialDrawing, head, body, leftArm, rightArm, leftLeg,
rightLeg };
};
const drawMan = (count) => {
let { head, body, leftArm, rightArm, leftLeg, rightLeg } =
canvasCreator();
switch (count) {
case 1:
head();

```

```
break;
case 2:
body();
break;
case 3:
leftArm();
break;
case 4:
rightArm();
break;
case 5:
leftLeg();
break;
case 6:
rightLeg();
break;
default:
break;
}
};
newGameButton.addEventListener("click", initializer);
window.onload = initializer;
```

2 Appendix B: User Manual

2.1 Starting:

1.You can start the game by clicking on the PLAY GAME button.

2.To see the rules you can click the RULES button.

2.2 Game Progress:

- It will record your right and wrong guesses.
- The objective of the game is to guess the word before the gallows are completed.
- Guess the word correctly to win before the gallows are completed, or else you will lose.

2.3 Understanding Feedback:

- Correct Guess: If a letter has been correctly guessed, the underscores are substituted with the right letter.
- Losing Guess: If you have guessed the wrong letter, then the game draws in one part of the hangman's gallows.